

Heritage Preservation AI

A Computer Vision and RAG-Based System for the National Heritage Preservation Trust

Module: Machine Learning and Related Applications

Assessment | CW - 01

National Institute of Business Management — National Innovation Center, Department of Data Science

Amani Mahroof - COBSCDS252F-013

Github repository link - https://github.com/Aanimahroof/Heritage_AI

Part A – System Design & Ethics

A1. System Architecture

1. Architecture Diagram

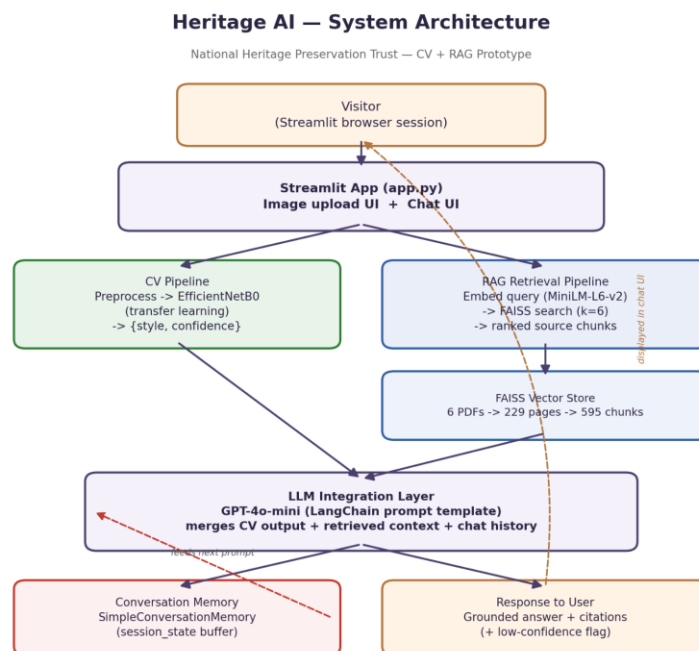


Figure 1. End-to-end data flow: the Streamlit app captures either an uploaded image or a chat question; the CV pipeline and RAG retrieval pipeline run independently and merge inside the LLM integration layer, which produces a cited answer while updating per-session conversation memory.

2. Technology Justification

CV Model Choice

In choosing the architecture, the option of EfficientNetB0 has been considered due to its good balance between performance and portability since, as an architectural style classifier, this model can provide decent classification accuracy while having the compact design, being approximately 5 million parameters large and thus able to be run on standard CPU hardware without the need of GPUs in the deployment stage after transfer learning from the pretrained on ImageNet model. The reason is the necessity of the efficient classification of multiple site visitors' images within the NHPT case study.

Transfer learning has been used since the given dataset, consisting of 2,331 images in five classes, is relatively small from the perspective of deep learning. Transfer learning makes use of the features learned in advance through millions of images from ImageNet to specialise the model to the architectural style problem with much less data than training a network from scratch, and, therefore, decreases the possibility of overfitting to this dataset. The only thing to be done is to replace the top classification layer.

LLM Choice

GPT-4o-mini was chosen for the conversational and explanatory parts of the system. The choice was mostly motivated by the cost factor since the model is significantly cheaper than other models within its family but

still capable of performing retrieval augmented question answering task. The prompt clearly allows for using the model as an acceptable option along with GPT-4 and similar models. As a prototype meant to be shown to the NHPT to prove its feasibility rather than handle actual requests from the start, saving on costs without compromising the quality seemed like a more important goal compared to maximising the capabilities of the underlying model. The relatively low temperature (0.2) was used everywhere in order to produce grounded and coherent results rather than imaginative ones.

Why LangChain

LangChain was chosen in order to connect the pieces of the system which would have required much custom coding to be connected: the retrieval chain from the vector store of FAISS to the model, prompt template management to structure the way the documents returned by retrieval and CV output will be formatted and sent to the model, and initially, the conversational memory component. It was found out that LangChain memory classes (e.g., ConversationBufferMemory) are unreliable and raise `ModuleNotFoundError` in recent LangChain versions; thus, the memory piece was substituted by a small and autonomous `SimpleConversationMemory` class that accomplishes the same purpose: it stores Q&A pairs and formats them to the prompt without the help of any LangChain component that is prone to changes. LangChain document loaders, text splitters, prompt templates, and the FAISS embeddings were retained due to their stability and the reduction of custom code required.

RAG Design Decisions

FAISS was chosen to be the vector database over other available options, like Chroma, or services like Pinecone, based on its light and serverless nature: running the knowledge base consisting of six documents does not require any external services, network delays, or hosting fees, which is suitable for the prototype on this level. The `RecursiveCharacterTextSplitter` was used to chunk the source PDF files (chunk size of 1,000 characters and overlap of 200 characters), because it divides the text along natural boundaries, i.e., paragraphs, and then sentences, instead of arbitrary character count, and therefore less likely to chop a sentence in two. This decision was made based on an experience from earlier stages of the project, when the candidate source document turned out to be truncated preview text instead of full paragraphs, and got replaced before the vector store was created to avoid a problem of poor quality retrieval that would be harder to detect after this step. The `all-MiniLM-L6-v2` sentence-transformer was chosen for creating embeddings based on its relatively small size (around 80MB) and ability to perform inference fast on the CPU, thus avoiding GPU dependencies in the retrieval pipeline, just like the CV model does.

Scalability & Cost Considerations

The cost structure of this system breaks down neatly into the one-time cost of training (CV model trained only once on the free GPU tier of Colab), as well as the recurrent cost of inference (CPU CV inference, local FAISS search and individual LLM API requests). This way, the marginal cost of each additional user query is almost entirely due to the LLM API call, and the operation cost remains predictable and relatively low compared to systems that rely on having the GPU infrastructure running persistently in order to handle every request. In case the corpus of documents or the number of users behind NHPT exceeds the prototype's capabilities, say, if the number of documents grows from six to several hundred to include all heritage sites overseen by the Trust, the FAISS vector search can be easily replaced by Pinecone or other similar hosted vector databases without making changes to the rest of the LangChain system (as the retrieval interface, `similarity_search` remains the same).

A2. Ethical & Risk Analysis

GDPR & Data Handling

Two types of data that can be considered sensitive are uploaded images and chat logs. Heritage site images uploaded to the system cannot contain any personal data in most cases, although capturing the image of visitors' faces or their license plate number could happen incidentally; this case needs to be considered as containing personal data under GDPR. Having a retention policy — for instance, deleting an uploaded image immediately after classification directly from server memory instead of keeping it forever — would significantly lower this risk, which is quite easy to do because of how the current system architecture works.

However, apart from that, the conversation logs saved through SimpleConversationMemory, the component that enables multi-turn conversations, bring up one more thing: even without revealing any personally identifying information, the conversation itself might provide hints about the visitor's interests, location, and habits while using the website. In fact, given the current implementation of conversation logging, all the logs are stored only in the session_state of Streamlit, which is cleared upon closing the session in a web browser or pressing the "reset" button, which means that at the moment none of the conversation logs are saved between visits.

Model Bias

The bias was discovered and fixed at the computer vision stage. There is class imbalance within the training set, with the most represented class (Victorian, around 720 samples) being around 2.4 times bigger than the least represented one (Roman, around 301 samples). Without fixing that, it would cause the system to underrepresent certain architectural styles in a way that would lead to underperformance on those particular architectures in the model. It has been mitigated via weighted cross entropy loss while training that makes the cost of underrepresentation greater for the minority classes. The mitigation was successful because despite being the least represented class, the Roman architecture got 0.90 recall value in the test set.

Another important aspect related to potential biases refers to the nature of the dataset on which the analysis has been performed: since there is no dataset with precisely five heritage styles, the labels have been transformed into the closest existing ones available in the source dataset. Thus, Gothic and Baroque labels remained the same; Roman has been translated into Romanesque architecture; Victorian into Queen Anne architecture; Neoclassical into Greek Revival architecture. Consequently, when interpreting the results of classification performed by the system, it is necessary to keep in mind that the classification refers to the mentioned proxy categories, and not to the precise definition of the heritage style, which, however, should be clearly explained to the user of the system rather than advertised as exact categorisation, as a visitor will likely interpret the term 'Roman' as the style of true Roman architecture. The bias associated with the LLM (inherent to any training data for the language model) has not been separately audited within this project but should be noted as an issue that needs to be monitored in any production application, especially regarding the cultural and religious significance of the architectural tradition.

Hallucination Risk

Retrieval augmentation is likely to mitigate but does not prevent hallucinations entirely. Having to base the generated response on the retrieved information serves as an anchor to prevent the model from creating an

answer out of thin air based on the training data alone, but LLMs can still misunderstand or augment the retrieved information, especially in cases of low-relevance retrieval when there simply is no relevant document for the user's query. It could be seen firsthand during testing, since inquisitive queries loaded with pronouns ('What are its most unique features?' following a question about Gothic architecture) sometimes yielded irrelevant document chunks because of how similarity search only encodes the raw question itself and not resolves 'it' to 'Gothic architecture'.

These mitigation measures incorporated in this system are the inclusion of an explicit citation of sources for all answers provided by the system, where users can independently confirm the veracity of the claims from the source document and not blindly accept the results of the system, and prompt-level instructions that would compel the model to acknowledge its limitations in cases when it does not have enough information from the retrieved context. In every instance where the system failed to generate an answer, it did so rightly by refusing to provide any answer.

Transparency

Transparency is ensured through two tangible means. Firstly, the confidence scores of the CV model were assessed for true information value rather than taken as an assumption: the testing on individual examples demonstrated that the images incorrectly classified by the model had significantly lower confidence scores (46-73%) compared to correctly classified images (frequently 98-100%), which proves that the confidence score is indeed a signal of the uncertainty of the model. Thus, the choice of the interface that shows the confidence scores of images to the users through the colored confidence bar, and a warning message when the confidence level is less than 60% has been confirmed as a good design decision. Secondly, citations to the RAG – the source filename and the page number of each retrieved chunk – enable traceability of the LLM-generated answers, making it possible for the user to check whether the claims made by the system are correct based on the heritage documents.

Mitigation Strategies (Summary)

- The images uploaded are safe, as they are processed in memory (not stored on disk) and conversation history remains in Streamlit's session_state. An UI policy statement for this aspect has not been made yet, but might be included later in the future.
- Class Imbalance problem has been tackled using Cross Entropy Loss with class weighting and has worked well (Roman class: recall 0.90 despite the lowest number of images trained). It should be noted here that proxy label problem in the dataset (e.g., Roman class tagged as Romanesque buildings) is discussed in the report but not the app.
- Apply loss functions to account for class imbalance within the training dataset, and make the restrictions associated with proxy-labels in the base dataset clear to the user.
- Base the answers generated by LLMs on the retrieved document and cite sources for each one.
- Present CV confidence levels to users, while explicitly warning users about predictions that fall below a certain threshold level of confidence.
- Configure the retrieval process such that the system will not answer at all if there is no relevant document rather than generating a fake answer, and interpret any such refusal to answer as an indication of retrieval failure.

Part B – Computer Vision Implementation

Option B2 (Architectural / Artifact Classification) was selected. EfficientNetB0 was fine-tuned via transfer learning to classify five architectural styles — Baroque, Gothic, Neoclassical, Roman, and Victorian — from a dataset of 2,331 images sourced from the Kaggle 'Architectural Styles Dataset' (based on Xu et al., 2014).

Training & Evaluation Summary

- Dataset: 2,331 images across 5 classes, split 70% train (1,631) / 10% validation (233) / 20% test (467), fixed random seed = 42 for reproducibility.
- Class imbalance addressed with weighted cross-entropy loss (training counts ranged from 214 for Roman to 492 for Victorian).
- Early stopping monitored validation loss (patience = 5 epochs); training stopped at epoch 10, with the best checkpoint at epoch 5 (validation loss 0.2035) reloaded for final evaluation.
- Final test accuracy: 91.9% (467 held-out test images), with per-class F1-scores ranging from 0.87 (Roman) to 0.95 (Victorian).

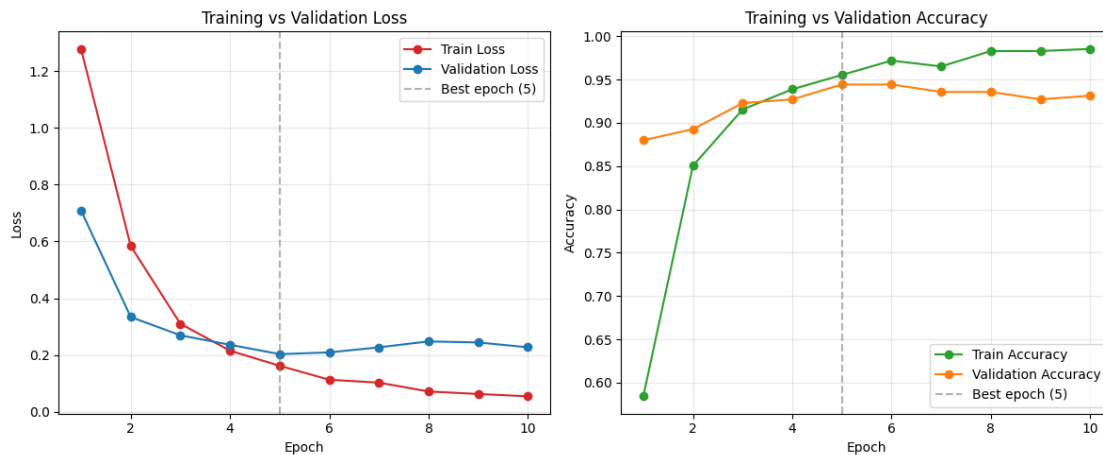


Figure 2. Training and validation loss/accuracy across epochs. Validation loss reaches its minimum at epoch 5 before beginning to rise, correctly triggering early stopping at epoch 10.

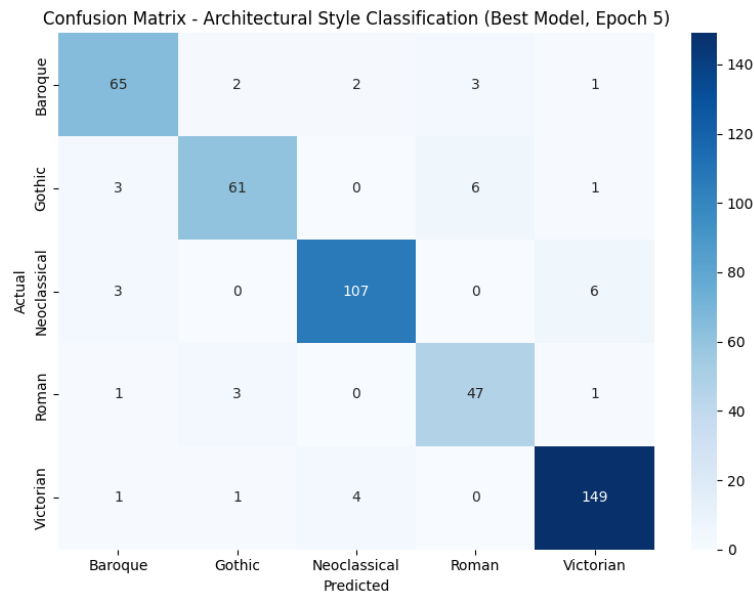


Figure 3. Confusion matrix on the 467-image test set (best checkpoint, epoch 5).

	precision	recall	f1-score	support
Baroque	0.89	0.89	0.89	73
Gothic	0.91	0.86	0.88	71
Neoclassical	0.95	0.92	0.93	116
Roman	0.84	0.90	0.87	52
Victorian	0.94	0.96	0.95	155
accuracy			0.92	467
macro avg	0.91	0.91	0.91	467
weighted avg	0.92	0.92	0.92	467

Figure 4. Classification report over every class.

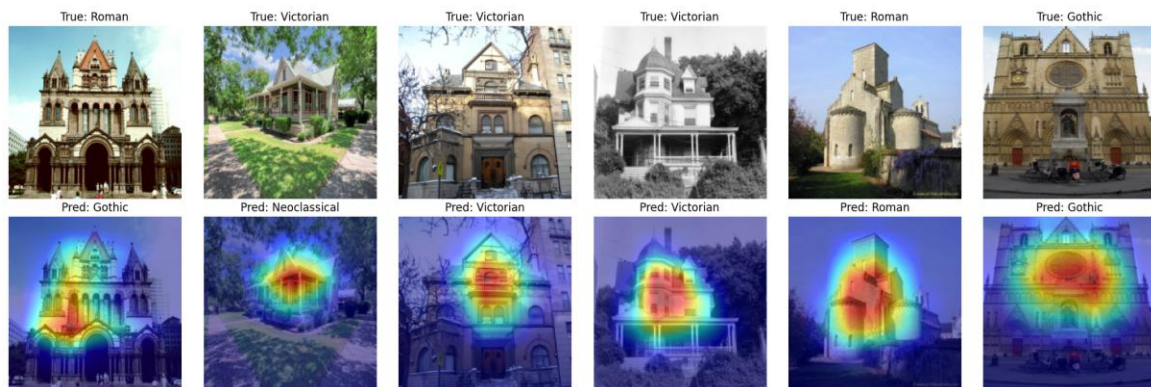


Figure 5. Grad-CAM visualisations on six test images: the model attends to genuine architectural features (facades, arches, rooflines) rather than background.

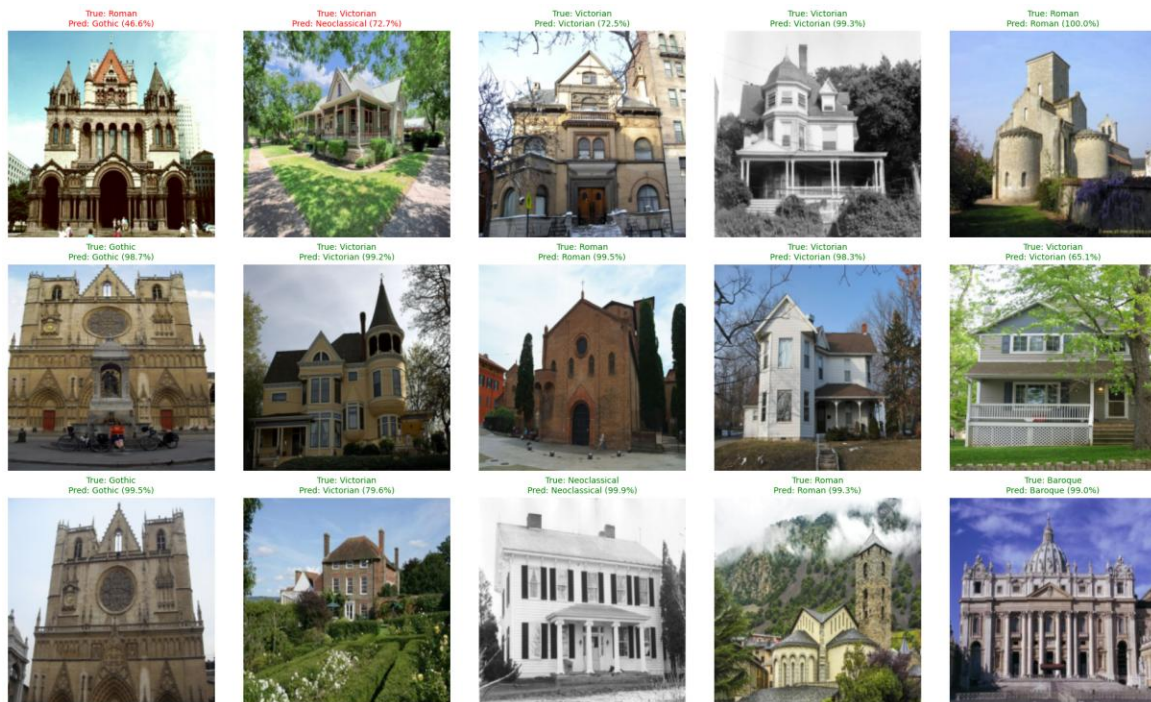


Figure 6. 15 example test predictions, with true label, predicted label and confidence shown (green = correct, red = incorrect).

Design Choices and Limitations

As a choice for architectural style classification through transfer learning, EfficientNetB0 was chosen since it strikes an appropriate balance between accuracy and efficiency. In particular, it is small enough for deployment but makes use of generally learned visual features due to the pretraining on ImageNet. The images were downloaded from the Kaggle Architectural Styles Dataset, (which is based on Xu et al., 2014). It is worth noting that this dataset does not include the labels according to the specific names of architectural styles that we are required to use. In other words, the following mapping has been used for our five classes: the Gothic and Baroque styles correspond to their names exactly; the Roman style corresponds to Romanesque architecture; the Victorian style corresponds to Queen Anne architecture; and the Neoclassical style corresponds to Greek Revival architecture.

Class imbalance was present in the dataset (492 images of Victorian architecture and 214 images of Roman architecture in the train set, ratio of 2.3). This was dealt with through the use of weighted cross entropy loss function and was successful since Roman, which had the least number of images in the train set, got a 0.90 recall score.

Random horizontal flips, rotations (-10° to $+10^\circ$), and contrast/brightness jitter were performed on training images to improve generalization while no transformations were done on validation and testing data sets.

A first attempt with 80/20 split between train and test data using early stopping based on training loss resulted in 99.2% training accuracy but just 92% test accuracy, suggesting over-fitting not caught by training loss. The right methodology involved a 70/10/20 split between train, validation, and test, using early stopping on the basis of validation loss. Epoch 5 was correctly marked as the point where the model achieved its peak generalization performance, as after this point, validation loss started increasing while the training loss kept improving. The best checkpoint was loaded again to find out 91.9% test accuracy.

However, the Grad-CAM visualisations revealed that the model focused on real architecture attributes such as facades, arches, roof lines, etc. The confidence levels were indicative of how wrong or right a

classification was: misclassification had a confidence level of 46-73%, while classifications that were right had 98-100%. This is important as the model's uncertainty estimation is valuable information which was passed down to the downstream LLM part in Part C.

Part C – LangChain Conversational System

Implementation Summary

Document Loading. The six PDF documents related to heritage architecture (Baroque, Gothic, Neoclassical, Romanesque, Victorian / Queen Anne and Conservation and Restoration of Cultural Property) have been loaded using PyPDFLoader resulting in 229 pages and 429,293 characters of source document without any truncation, thus reaffirming the decision of using a different source document (previously used UNESCO exhibit page which had only truncated text).

Vector Store. Documents were chunked by RecursiveCharacterTextSplitter (chunk size 1,000, overlap 200), which resulted in 595 chunks, averaging 812 characters per chunk. These chunks were embedded locally by all-MiniLM-L6-v2 model (free, runs on the CPU without any API charges) and indexed using FAISS, saving to the disk so that embedding is not needed on each run. The sanity check of retrieval showed that vector store works well with distinguishing styles: query for Gothic architecture gets chunks from Gothic sources, query for Baroque architecture gets chunks from Baroque sources.

RAG Chain with Citations. The retrieval process for each user query is the extraction of the top 6 most relevant chunks, where each chunk contains an explicit source file name along with page number to be fed into GPT-4o-mini (with a temperature setting of 0.2 because lower values favor consistency over creativity). The prompt used explicitly instructs the model to use only the context available, to mention if the context does not suffice for answering a question, and to cite its sources in a specific format. This yielded properly grounded and cited answers for direct questions like 'What is Gothic architecture?'.

CV to LLM Integration. The EfficientNetB0 model is loaded and used to do inference on the CPU, and the structure of the output ({style, confidence}) is provided to the specific prompt template which requests the LLM to give an explanation of the predicted style with the help of the retrieved context. The user should be notified about the uncertainty in the prediction if the confidence is less than 60%. An example of test image generated Gothic architecture with the confidence of 98.68%, along with its explanation.

Multi-turn Memory. The SimpleConversationMemory stores each question and answer as well as the conversation history that is being input to the next prompt. Six examples of conversations (three to four turns each) including Gothic architecture, Baroque style, Neoclassical architecture, Romanesque architecture, Queen Anne architecture and heritage conservation have been simulated as transcripts in RAG_Pipeline.ipynb

Prompt Design, RAG Configuration, Hallucination Mitigation & CV-LLM Handoff

Design of prompts distinguishes between three different templates for three different tasks: one for direct question asking in RAG, another for image prediction and explanation using structured output of the CV model, and finally, one for multi-turn chat that takes into account conversation history. All three are explicitly designed to make the model provide answers only based on the available context, and decline to fabricate something if there is not enough context available.

The RAG architecture is based on FAISS, using all-MiniLM-L6-v2 embeddings, which have zero API costs and are faster in CPU inference locally. The index consists of 595 chunks obtained from 6 documents, returning 6 chunks for each query.

There are two ways of dealing with hallucinations in our system: instruction to avoid guessing at the prompt level and citation rules that ensure that every claim made must be referenced to a particular document and its page. One interesting and true discovery was made while testing: in case of multi-turn follow-ups when pronouns ("its", "this style", "it") were used without the rephrasing of the subject, the process of retrieval that

relies solely on the raw text of the question did not manage to find the right documents because it cannot use the previous turns of the dialogue in order to interpret pronouns. In these cases the system did not provide an answer and said that it could not do so with the given context, instead of providing some fake information. This is indeed a problem with the quality of retrieval for follow-ups that contain pronouns, however, this shows that our design works exactly as intended in a case of failure: the system fails gracefully.

The CV-LLM Data Transfer is where the unaltered dictionary containing the structured prediction produced by the CV model ({style, confidence}) goes straight into a unique prompt template that uses the style name prediction to find context and tell the LLM to explicitly point out predictions below the 60% confidence level.

Example Conversation Excerpts

Two representative excerpts are reproduced below; all six full transcripts (3–4 exchanges each) are captured in RAG_Pipeline.ipynb and included with the submission.

Excerpt 1 — Gothic architecture, demonstrating pronoun resolution and a safe retrieval failure:

```
Turn 1
User: What is Gothic architecture?
Assistant: Gothic architecture is a style of architecture that originated in the early 12th century in northwest France and England, spreading througho

Sources: [Gothic_architecture.pdf, page 0], [Gothic_architecture.pdf, page 3]
(Retrieved from: Gothic_architecture.pdf)
=====
Turn 2
User: What are its most unique characteristics?
Assistant: The reference context provided does not contain specific information about the unique characteristics of Gothic architecture. Therefore, I c

Sources: None
(Retrieved from: Romanesque_architecture.pdf, Conservation_and_restoration_of_cultural_property.pdf, Neoclassical_architecture.pdf)
=====
Turn 3
User: Give me 3 real examples of buildings in this style.
Assistant: The reference context provided does not contain specific examples of buildings in the Gothic architectural style. Therefore, I cannot provid

Sources: None
(Retrieved from: Romanesque_architecture.pdf, Baroque_architecture.pdf, Neoclassical_architecture.pdf)
=====
Turn 4
User: How does it compare to Baroque architecture?
Assistant: The reference context provided does not contain specific information comparing Gothic architecture to Baroque architecture. Therefore, I can

Sources: None
(Retrieved from: Baroque_architecture.pdf, Neoclassical_architecture.pdf)
=====
```

Excerpt 2 — Heritage conservation, demonstrating grounded multi-turn answering:

```
Turn 1
User: What does conservation and restoration of cultural property involve?
Assistant: Conservation and restoration of cultural property involve the protection and care of cultural property, which includes artworks, architectur

Sources: [Conservation_and_restoration_of_cultural_property.pdf, page 0, page 5]
(Retrieved from: Conservation_and_restoration_of_cultural_property.pdf)
=====
Turn 2
User: Why is this important for heritage sites specifically?
Assistant: The conservation and restoration of cultural property, particularly for heritage sites, is crucial because it ensures the protection and pre

Sources: [Conservation_and_restoration_of_cultural_property.pdf, page 0, page 12]
(Retrieved from: Conservation_and_restoration_of_cultural_property.pdf)
=====
Turn 3
User: What are common challenges in restoring historic buildings?
Assistant: The reference context provided does not specifically address the common challenges in restoring historic buildings. Therefore, I cannot prov

Sources: None
(Retrieved from: Romanesque_architecture.pdf, Conservation_and_restoration_of_cultural_property.pdf, Gothic_architecture.pdf)
=====
```

#	Topic	Exchanges	Notes
1	Gothic architecture	4	Pronoun-reference retrieval gap observed
2	Baroque architecture	3	Correctly grounded throughout
3	Neoclassical architecture	3	Correctly grounded throughout
4	Romanesque architecture	3	Compares Romanesque vs. Gothic correctly
5	Victorian / Queen Anne style	3	One honest 'insufficient context' decline
6	Heritage conservation (general)	3	Correctly grounded throughout

Table 2. Summary of the six multi-turn example conversations (full transcripts in RAG_Pipeline.ipynb).

Part D – Technical Report, Testing & Documentation

1. System Overview

The developed system is a single page Streamlit app (app.py), consisting of two intertwined processes that share the same session state. The process on the left accepts the building image from the visitor; an already loaded EfficientNetB0 model classifies the image into one of the five architectural styles, showing it to the user with the confidence bar colored according to the confidence level (warning if less than 60%). The process on the right provides a chat interface where the visitor can pose their free-text questions; for each question, the top 6 similar chunks (according to the similarity scores in the FAISS index) among six heritage PDFs are loaded along with the conversation history and the latest CV prediction (when available) into GPT-4o-mini and receive a cited and grounded answer. Streamlit rerun-on-every-interaction model is used through the implementation of `st.session_state` to keep chat history and current prediction, and `@st.cache_resource` to load CV model, vector store, and LLM client once instead of doing that on each keystroke/upload.

2. Implementation Challenges

CV difficulties:

- There was silent data loss due to failure in uploading to Google Drive (the Gothic class reduced from 331 to 27 images); this problem was solved through diagnosing and analyzing the difference between file counts locally and in cloud prior to training.
- Class imbalance became evident after initial data analysis; and to solve this problem, it needed to be mitigated through adopting weighted loss, rather than continuing with the unweighted training as default.
- The early stopping technique, which relied on monitoring training loss, could not identify any overfitting; this was identified only through analyzing the discrepancy between training and testing accuracy; it necessitated splitting data into 70/10/20 for train/validation/test and re-training with validation loss based early stopping.

LLM / RAG issues:

- The initial source document (from an UNESCO exhibit-panel web page) had just a preview text excerpt, not complete paragraphs; this has been detected and corrected prior to building the vector store, thus preventing the emergence of a difficult-to-diagnose retrieval quality issue.
- Multi-turn conversation tests indicated that the retrieval module does not use the history of conversation while embedding the current question, leading to problems with retrieval of documents related to pronoun-laden follow-up questions ('its', 'this style'). It was not discovered during single-turn tests, but came to light when multi-turn conversations were tested, highlighting the importance of testing multi-turn conversations.
- The pre-built ConversationBufferMemory class of LangChain produced an error of ModuleNotFound for the installed version of LangChain; this was corrected by using another class, called SimpleConversationMemory, to do the same job.

Integration challenges:

- Once constructed, the transition from CV to LLM flowed well, as the {style, confidence} was formatted correctly to be inserted into the prompt template.
- Connecting the CV panel and chat panel into one Streamlit session had some challenges regarding `st.session_state` because of how Streamlit works: Every time any widget is used, the whole script gets executed again.

3. Testing & Validation

Unit testing — CV preprocessing: the resize/normalise/augmentation pipeline was verified by inspecting output tensor shapes and confirming augmentation was applied only to the training split, not to validation or test data.

```
# Training transform: resize + augmentation (flip/rotate/color) + normalize
train_transform = transforms.Compose([
    transforms.Resize((224, 224)), # resize to EfficientNetB0's required input size
    transforms.RandomHorizontalFlip(), # randomly mirror image to improve generalization
    transforms.RandomRotation(10), # randomly rotate up to 10 degrees to simulate tilt
    transforms.ColorJitter(brightness=0.2, contrast=0.2), # randomly vary lighting/contrast
    transforms.ToTensor(), # convert image to a PyTorch tensor
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
]) #converts the image from a picture format into numbers [normalise using imagenet stat]

# Test transform: resize + normalize only, no augmentation (for fair evaluation)
test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
])

# Load the same image folder twice, once per transform pipeline [with augmented data and real data]
full_dataset_train = datasets.ImageFolder(root=dataset_path, transform=train_transform)
full_dataset_test = datasets.ImageFolder(root=dataset_path, transform=test_transform)

print("Classes found:", full_dataset_train.classes)
print("Total images:", len(full_dataset_train))

Classes found: ['Baroque', 'Gothic', 'Neoclassical', 'Roman', 'Victorian']
Total images: 2331
Train size: 1631
Validation size: 233
Test size: 467
```

Unit testing — retrieval pipeline: the vector store was confirmed to distinguish correctly between architectural styles in sanity-check queries (Gothic vs. Baroque queries return non-overlapping, style-correct sources), and document loading was confirmed complete and non-truncated (429,293 characters across 229 pages, no missing content).

```
Found 6 PDF(s) in ..\Documents:
- Baroque_architecture.pdf
- Conservation_and_restoration_of_cultural_property.pdf
- Gothic_architecture.pdf
- Neoclassical_architecture.pdf
- Romanesque_architecture.pdf
- victorian_Queen_Anne_architecture.pdf
Loaded 'Baroque_architecture.pdf': 34 page(s)
Loaded 'Conservation_and_restoration_of_cultural_property.pdf': 23 page(s)
Loaded 'Gothic_architecture.pdf': 72 page(s)
Loaded 'Neoclassical_architecture.pdf': 38 page(s)
Loaded 'Romanesque_architecture.pdf': 57 page(s)
Loaded 'victorian_Queen_Anne_architecture.pdf': 5 page(s)

--- Sanity check ---
Total pages loaded: 229
Total characters loaded: 429,293

--- First page preview (first 400 characters) ---
Baroque architecture
Clockwise from top left: Church of Saint Ignatius
of Loyola in Italy, Church of Santa Prisca de
Taxco in Mexico, Smolny Cathedral in Russia,
St-Gervais-et-St-Protais in France
Years active Late 16th-18th centuries
```

Model evaluation: 91.9% CV test accuracy on 467 held-out images, with per-class F1-scores documented in the classification report, and failure-case analysis via Grad-CAM showing misclassifications cluster along historically related style pairs (e.g. Roman/Gothic confusion) rather than occurring randomly.

	precision	recall	f1-score	support
Baroque	0.89	0.89	0.89	73
Gothic	0.91	0.86	0.88	71
Neoclassical	0.95	0.92	0.93	116
Roman	0.84	0.90	0.87	52
Victorian	0.94	0.96	0.95	155



Test Accuracy (best model, epoch 5): 0.9186

Failure case analysis — RAG: multi-turn conversations exposed a genuine retrieval limitation: pronoun-referenced follow-up questions frequently failed to retrieve the correct source documents, since similarity search embeds only the raw current question rather than a context-resolved version of it. In every observed failure case, the system responded honestly that it could not answer based on the available context rather than fabricating a plausible but ungrounded answer, confirming that the hallucination-mitigation prompt design holds up under a real, unplanned failure mode.

```
Q: what is gothic architecture?
A: Gothic architecture is a style that began in the early 13th century in northern France and England, characterized by features such as pointed arches, ribbed vaults, and flying buttresses.
Sources: [Gothic_architecture.pdf, page 4], [Gothic_architecture.pdf, page 1]
Retrieved from: [Gothic_architecture.pdf]

Q: How similar is gothic architecture to neoclassical architecture?
A: Examples of Baroque architecture include the Church of Saint Ignace of Loyola in Italy, the Church of Santa Prisca in Mexico, and the Church of the Gesù in Rome.
Sources: [Baroque_architecture.pdf]
Retrieved from: [Baroque_architecture.pdf]

Q: How different is neoclassical architecture from gothic architecture?
A: The context provided does not contain specific information comparing Neoclassical architecture and Gothic architecture. Therefore, I cannot provide a direct comparison.
Sources: [Neoclassical_architecture.pdf]
Retrieved from: [Neoclassical_architecture.pdf]
```

tell me about victorian architecture

I'm sorry, but the reference context provided does not contain information about Victorian architecture. Therefore, I cannot answer your question.

Sources: []

Sources: Gothic_architecture.pdf, Neoclassical_architecture.pdf

Uploaded image

Neoclassical

Confidence: 38.0%

Confidence is fairly low — treat this prediction with caution.

Reset conversation

Integration testing — CV to LLM workflow: validated end-to-end with a real test image (Gothic prediction at 98.68% confidence, correctly triggering a grounded, cited explanation referencing the predicted style).


```
Predicted style: Gothic
Confidence: 98.68%

--- Grounded, cited explanation ---
Gothic architecture is a prominent architectural style that flourished in Europe from the late 12th century to the 16th century, particularly during the 13th and 14th centuries.

Key elements of Gothic architecture include:

1. **Pointed Arches**: These arches not only provide structural support but also allow for taller and more slender designs compared to the rounded arch.
2. **Ribbed Vaults**: This technique involves intersecting stone ribs that support the ceiling, enabling the creation of expansive interior spaces.
3. **Flying Buttresses**: These external supports transfer the weight of the roof and walls outward, allowing for higher ceilings and larger windows.
4. **Large Stained Glass Windows**: Gothic buildings often feature intricate stained glass windows that fill the interiors with colorful light, depicting religious scenes.
5. **Ornate Decoration**: The style is known for its elaborate sculptures and detailed facades, often adorned with figures and motifs that convey religious themes.

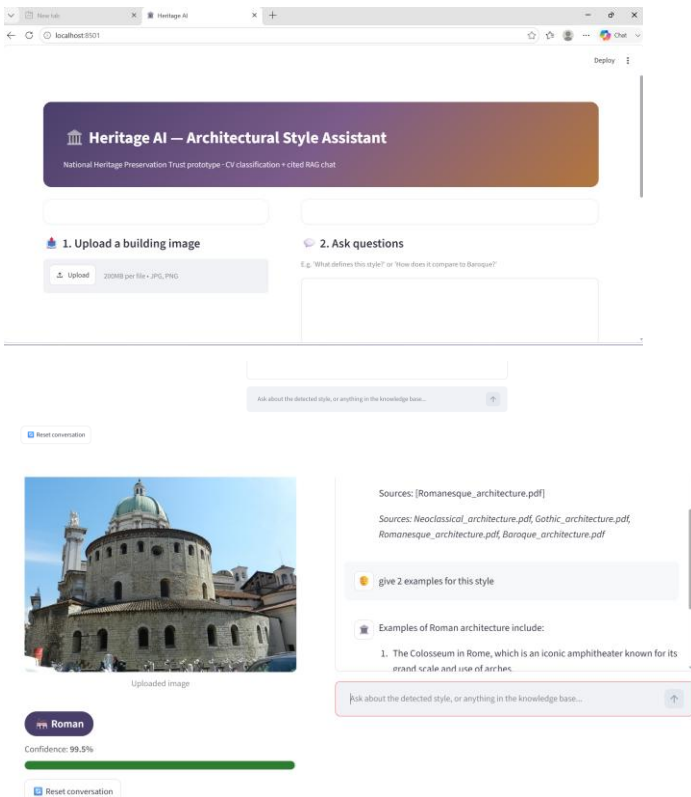
Gothic architecture is most famously represented in cathedrals and churches, with notable examples including Notre-Dame de Paris and Chartres Cathedral.

The term "Gothic" was initially used in a derogatory sense during the Renaissance by those who favored classical architecture, but it has since come to denote a specific historical style.

Sources: [gothic_architecture.pdf, page 0], [gothic_architecture.pdf, page 3], [gothic_architecture.pdf, page 60]

(Retrieved from: gothic_architecture.pdf)
```

Streamlit application testing: the app was manually exercised end-to-end — image upload, style prediction with confidence display, and multi-turn chat all functioning together within a single session via `session_state`, with the reset button correctly clearing memory and prediction state between visits.



4. Future Improvements

- The reformulation of queries for multi-turn dialogues where queries are reformulated based on the conversation history prior to embedding/retrieval instead of embedding the queries as they are would directly solve the problem of pronoun resolution retrieval discovered through testing.
- Solving the problem of proxy labels in the CV dataset by obtaining samples that are actually labeled in order to avoid using the nearest proxies.
- Increasing the minority-class dataset (Roman and Gothic) in order to bridge the remaining class imbalance gap.
- the gap in class-imbalance after applying only weighted loss correction.
- Option B1 is to extend the CV model pipeline to damage detection in order to incorporate an additional use case for the Trust, along with the existing one, which involves style classification.
- Option A1 to host FAISS in a vector database (for example, Pinecone) in case of growth in the knowledge base and/or user base beyond prototyping.
- Model quantisation for quicker inference on CPU for visitors that take images with a mobile device on the site.